

Support Vector Machines

from Scratch

Line-by-line theoretical commentary on `svm_from_scratch.py`

Lecture 02 — Finance Course

This document is a companion to `svm_from_scratch.py` in the *finance-course* repository. Every code block from the script is reproduced verbatim and followed by the theory behind it: the underlying math, the connection to Igor Halperin’s *Supervised, Unsupervised and Reinforcement Learning in Finance* slides (NYU Tandon, 2017), the practical pitfalls, and a step-by-step derivation of every key result.

The slides are organised in three parts:

- **Part I** — primal SVR / SVC, ε -insensitive loss, soft margin C .
- **Part II** — KKT conditions, the dual problem, the support-vector expansion, convex optimisation.
- **Part III** — the kernel trick: features $\Phi(x)$ replaced by a kernel $k(x, x')$.

Visual aids. Each section that has a corresponding step in `svm_visualize.py` embeds the relevant figure. Blue DERIVATION boxes carry the full algebra wherever a result is stated without proof.

1. Mathematical recap

1.1 Primal soft-margin SVC (Part I)

The primal problem minimises the squared norm of the weight vector (equivalently, maximises the margin width $2/\|w\|$) plus a penalty C times the total slack:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (1)$$

$$\text{s.t.} \quad y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n. \quad (2)$$

Here $y_i \in \{-1, +1\}$, ξ_i is the per-sample slack, and $C > 0$ controls the trade-off between margin width and tolerance for margin violations.

1.2 Where does the margin width $2/\|w\|$ come from?

Derivation : geometric margin

The decision hyperplane is $\{x : \langle w, x \rangle + b = 0\}$. The two canonical margin hyperplanes are

$$H_+ : \langle w, x \rangle + b = +1, \quad H_- : \langle w, x \rangle + b = -1.$$

Pick a point x_+ on H_+ and a point x_- on H_- . Their projection onto the unit normal $\hat{n} = w/\|w\|$ gives signed distances $\langle \hat{n}, x_+ \rangle = (1 - b)/\|w\|$ and $\langle \hat{n}, x_- \rangle = (-1 - b)/\|w\|$. Subtracting,

$$\text{margin width} = \langle \hat{n}, x_+ \rangle - \langle \hat{n}, x_- \rangle = \frac{2}{\|w\|}.$$

Maximising the margin width is therefore the same as minimising $\|w\|^2$.

1.3 Lagrangian

Introducing multipliers $\alpha_i \geq 0$ on the margin constraints and $\mu_i \geq 0$ on the slack-non-negativity constraints $\xi_i \geq 0$:

$$L(w, b, \xi, \alpha, \mu) = \frac{1}{2} \|w\|^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i(\langle w, x_i \rangle + b) - 1 + \xi_i] - \sum_i \mu_i \xi_i. \quad (3)$$

We seek a saddle point: minimise over (w, b, ξ) , maximise over $(\alpha, \mu) \geq 0$.

1.4 Stationarity conditions

Derivation : stationarity

Take partial derivatives of L in (3) and set them to zero.

With respect to w :

$$\frac{\partial L}{\partial w} = w - \sum_i \alpha_i y_i x_i = 0 \quad \implies \quad \boxed{w = \sum_i \alpha_i y_i x_i}$$

With respect to b :

$$\frac{\partial L}{\partial b} = - \sum_i \alpha_i y_i = 0 \quad \Rightarrow \quad \boxed{\sum_i \alpha_i y_i = 0}$$

With respect to ξ_i :

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \quad \Rightarrow \quad \mu_i = C - \alpha_i.$$

Combined with the dual-feasibility constraints $\alpha_i, \mu_i \geq 0$:

$$\boxed{0 \leq \alpha_i \leq C}$$

1.5 Reducing the Lagrangian to a function of α only

Derivation : primal $\rightarrow$ dual

Plug the stationarity conditions back into (3) and simplify term by term.

Term 1. The squared-norm term, using $w = \sum_i \alpha_i y_i x_i$:

$$\begin{aligned} \frac{1}{2} \|w\|^2 &= \frac{1}{2} \langle w, w \rangle \\ &= \frac{1}{2} \left\langle \sum_i \alpha_i y_i x_i, \sum_j \alpha_j y_j x_j \right\rangle \\ &= \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle. \end{aligned}$$

Term 2. The slack terms collapse:

$$C \sum_i \xi_i - \sum_i \alpha_i \xi_i - \sum_i \mu_i \xi_i = \sum_i \xi_i \underbrace{(C - \alpha_i - \mu_i)}_{=0} = 0.$$

Term 3. The margin term:

$$\begin{aligned} - \sum_i \alpha_i y_i \langle w, x_i \rangle &= - \left\langle w, \sum_i \alpha_i y_i x_i \right\rangle = - \langle w, w \rangle = - \|w\|^2, \\ - \sum_i \alpha_i y_i b &= -b \sum_i \alpha_i y_i = 0 \quad (\text{by stationarity in } b). \end{aligned}$$

Term 4. The constant term $+\sum_i \alpha_i$ survives unchanged.

Putting it all together,

$$\begin{aligned} L^*(\alpha) &= \frac{1}{2} \|w\|^2 - \|w\|^2 + \sum_i \alpha_i \\ &= -\frac{1}{2} \|w\|^2 + \sum_i \alpha_i \\ &= \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle. \end{aligned}$$

So the dual problem is

$$\max_{\alpha} \quad \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j k(x_i, x_j) \quad (4)$$

$$\text{s.t.} \quad \sum_i \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C. \quad (5)$$

where we have already replaced $\langle x_i, x_j \rangle$ by a general kernel $k(x_i, x_j)$ in anticipation of §8.

1.6 The kernel trick (Part III)

Because (4) depends on the data *only* through pairwise dot products, we can replace every $\langle x_i, x_j \rangle$ by a Mercer-admissible kernel

$$k(x, x') = \langle \Phi(x), \Phi(x') \rangle_{\mathcal{H}} \quad (6)$$

for some implicit feature map $\Phi : \mathcal{X} \rightarrow \mathcal{H}$ into a reproducing-kernel Hilbert space $\mathcal{H}$. The script uses the simplest possible kernel — the **linear** one

$$k(x, x') = \langle x, x' \rangle = x^\top x',$$

which corresponds to $\Phi(x) = x$, the identity. Because Φ is finite-dimensional here, we can also recover the primal weight vector w explicitly — impossible with the Gaussian RBF kernel whose Φ lives in an infinite-dimensional space.

1.7 The support-vector expansion

Substituting $w = \sum_i \alpha_i y_i x_i$ into $f(x) = \langle w, x \rangle + b$ gives the slide's central formula:

$$f(x) = \sum_{i \in \text{SV}} \alpha_i y_i k(x_i, x) + b. \quad (7)$$

By KKT *complementary slackness* (derived in §10), only support vectors (those with $\alpha_i > 0$) contribute.

2. Imports

```
import cvxpy as cp
import numpy as np
import pandas as pd
from sklearn.metrics import roc_auc_score
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

cvxpy is a domain-specific language for convex optimisation. It lets us write the SVM dual as a literal mathematical expression and hand it to a convex QP solver (CLARABEL by default), bypassing the SMO algorithm libsvm uses internally.

numpy provides the array linear algebra: dot products, broadcasting, matrix multiplications. The Gram matrix $K = XX^\top$ is a single $\mathcal{O}$ operation.

pandas is used only to assemble the synthetic dataset with labelled columns — not strictly necessary for the math.

roc_auc_score is the rank-based area under the ROC curve. **train_test_split** is sklearn's stratified random splitter. **StandardScaler** standardises each feature column to mean 0 and standard deviation 1 — mandatory for distance-based kernels.

SVC is sklearn's reference implementation (calling libsvm under the hood); we train it with identical hyperparameters and verify the from-scratch solution matches.

3. Reproducibility

```
RANDOM_STATE = 42
rng = np.random.default_rng(RANDOM_STATE)
```

A single integer seeds every random draw downstream: borrower features, default labels, train/test split. Fixing the seed makes the from-scratch vs sklearn comparison deterministic.

4. Synthetic credit dataset

```
n_samples = 500

debt_to_income      = rng.beta(2, 5, n_samples) * 1.2
credit_utilization  = rng.beta(2, 4, n_samples)
missed_payments     = rng.poisson(0.6, n_samples)
savings_ratio       = rng.beta(2, 5, n_samples) * 0.5
income_volatility    = np.abs(rng.normal(0.15, 0.1, n_samples))
```

Five risk factors drawn from distributions chosen to roughly mimic a real retail-credit book: Beta distributions for bounded ratios, Poisson for the missed-payments count, and a truncated normal for income volatility.

4.1 Logistic data-generating process

```

logit = (
    -4.0
    + 8.0 * debt_to_income
    + 5.0 * credit_utilization
    + 2.0 * missed_payments
    - 8.0 * savings_ratio
    + 5.0 * income_volatility
)
p_default = 1.0 / (1.0 + np.exp(-logit))
y_01      = (rng.uniform(size=n_samples) < p_default).astype(int)
y_signed  = 2 * y_01 - 1

```

The default label comes from a known logistic model. Each risk factor contributes additively to the **log-odds**:

$$\eta_i = \beta_0 + \beta_1 x_{i,1} + \cdots + \beta_5 x_{i,5}.$$

The sigmoid then maps log-odds to probability:

$$p_i = \sigma(\eta_i) = \frac{1}{1 + e^{-\eta_i}} \in [0, 1].$$

Derivation : inverse-transform Bernoulli sampling

We want $y_i \sim \text{Bernoulli}(p_i)$ but only have $U \sim \text{Uniform}(0, 1)$ to draw from. The CDF of a Bernoulli with parameter p is the step function $F_p(t) = (1 - p) \cdot \mathbf{1}[t \geq 0] + p \cdot \mathbf{1}[t \geq 1]$, whose inverse is

$$F_p^{-1}(u) = \begin{cases} 0 & u \leq 1 - p \\ 1 & u > 1 - p \end{cases} = \mathbf{1}\{u > 1 - p\} = \mathbf{1}\{1 - u < p\}.$$

Since $1 - U$ is itself $\text{Uniform}(0, 1)$, define $y = \mathbf{1}\{U < p\}$. Then for any $p \in [0, 1]$,

$$\Pr(y = 1) = \Pr(U < p) = p,$$

which is the Bernoulli law. The single line `(rng.uniform(...) < p_default).astype(int)` implements this for the entire population vectorised.

The line `y_signed = 2 * y_01 - 1` converts labels from $\{0, 1\}$ to $\{-1, +1\}$ — the convention used throughout the slide math (where y_i multiplies the decision function $f(x)$).

Pitfall

If the logit coefficients are too weak, the soft-margin SVM degenerates to $w \approx 0$ (predict-all-majority) and the comparison to sklearn becomes noise-dominated. The coefficients above (8.0 on DTI, etc.) ensure a clean linear separation.

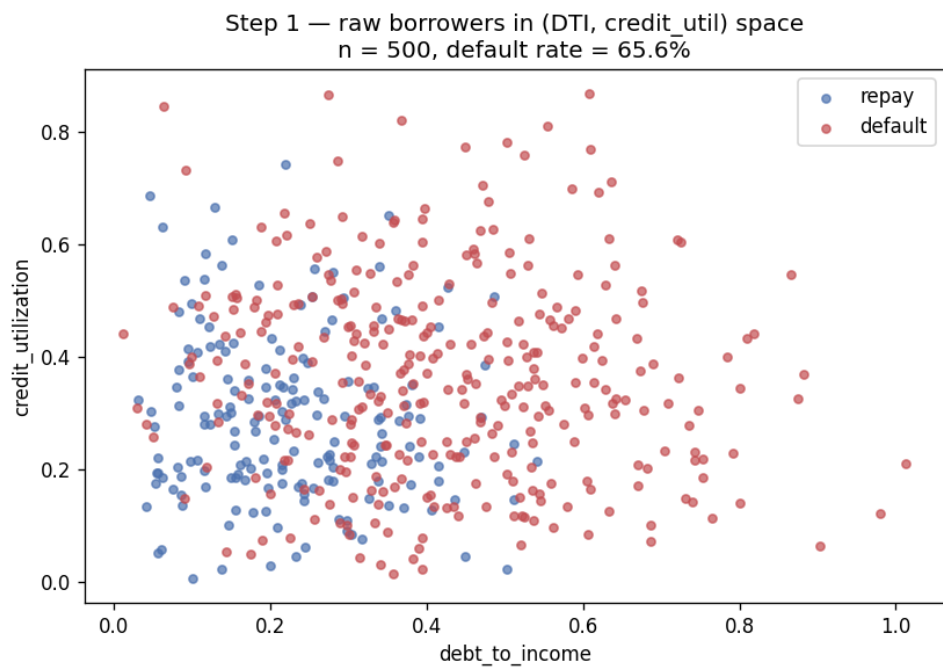


Figure 1. The synthetic credit book in the (debt-to-income, credit-utilisation) plane. Defaults concentrate in the upper-right; this visible separation is what makes the linear SVM viable.

5. Stratified train/test split

```
(X_train, X_test,
 y_train_01, y_test_01,
 y_train, y_test) = train_test_split(
    X_dataframe.values, y_01, y_signed,
    test_size=0.25, stratify=y_01, random_state=RANDOM_STATE,
)
```

`train_test_split` accepts any number of parallel arrays as positional arguments and splits them in lockstep using the same row indices. Three inputs (X , y_{01} , y_{signed}) produce six aligned outputs.

5.1 Stratification preserves class balance

Without stratification, the split is uniform over rows and the default rate in each half drifts from the marginal by Bernoulli sampling noise. With `stratify=y_01`, sklearn samples within each class separately, enforcing

$$\Pr(y = 1 \mid \text{train}) = \Pr(y = 1 \mid \text{test}) = \Pr(y = 1) \quad (\text{deterministically, not in expectation}).$$

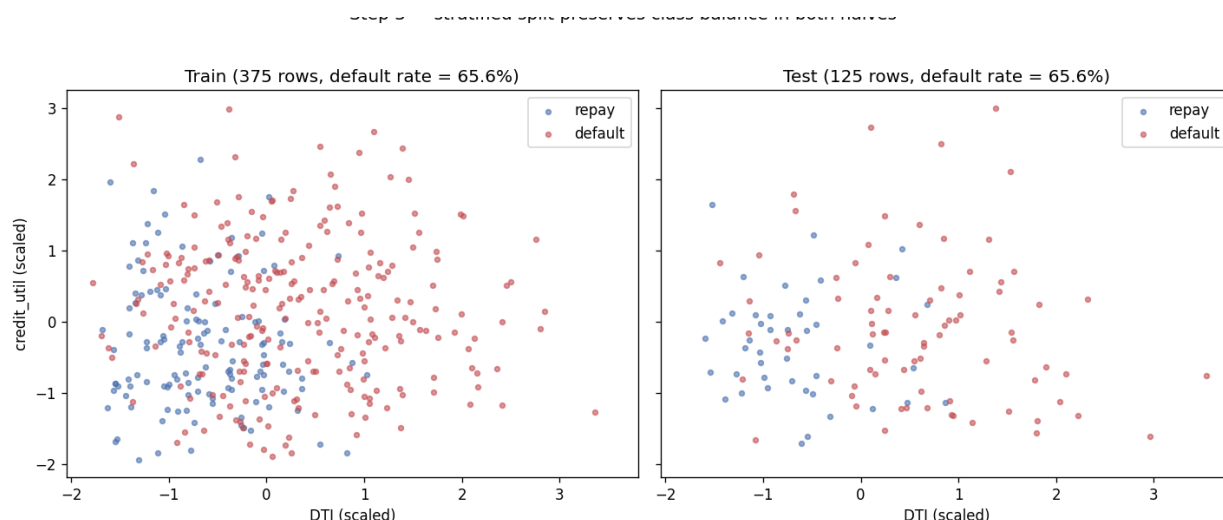


Figure 2. Stratified split. Both halves carry the same 65.6% default rate by construction (annotated in each subtitle).

Pitfall

Without stratification on small n , you can get unlucky splits like 70% / 50% defaults — making the test set look either too hard or too easy and biasing every metric.

6. Feature scaling

```
scaler      = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled  = scaler.transform(X_test)
```

Each column is remapped via the z-score transformation:

$$x'_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j},$$

where (μ_j, σ_j) are computed from the **training set only**.

6.1 Why scaling is mandatory for SVMs

Every kernel measures distances or dot products. With the linear kernel $k(x, x') = \sum_j x_j x'_j$, a column with range 1–40 swamps a column with range 0–1. After scaling, all columns contribute on the same numeric footing.

Derivation : why $\bar{X}'.\text{var}() = 1$ exactly after scaling

Treating the whole $X' \in \mathbb{R}^{n \times d}$ matrix as nd scalars, the pooled variance decomposes as

$$\text{Var}(X') = \underbrace{\frac{1}{d} \sum_j \text{Var}(X'_{:,j})}_{\text{mean within-column variance}} + \underbrace{\text{Var}(\bar{X}'_{:,j})}_{\text{variance across column means}}.$$

After **StandardScaler**, every column has variance exactly 1 and mean exactly 0, so the first term is 1 and the second is 0. The flattened variance is therefore 1 to machine precision.

Step 2 : Feature scaling: same shape, common units

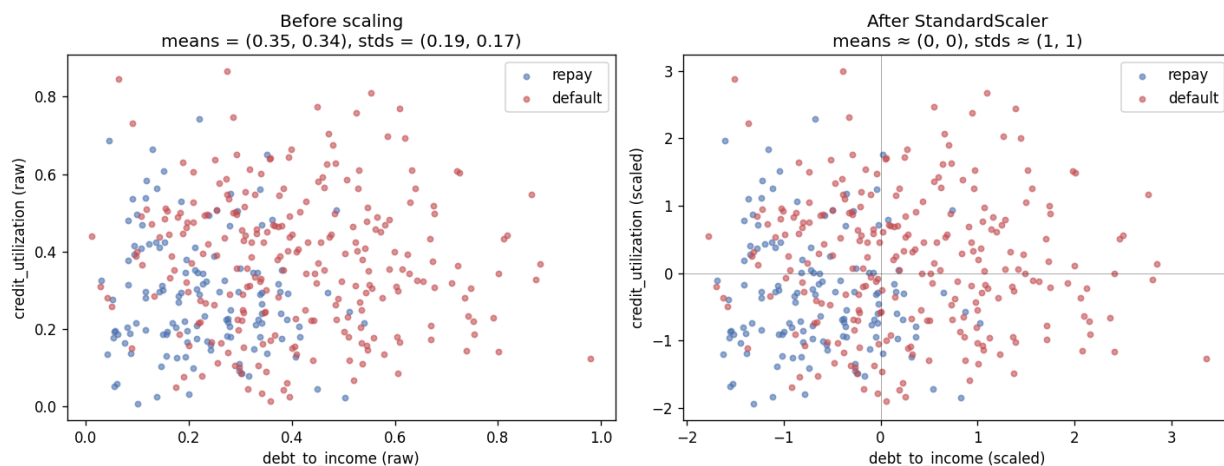


Figure 3. Before vs after **StandardScaler**. The point cloud has the same shape; only its location (mean) and scale (standard deviation) change.

Pitfall

Fit the scaler on **X_train** only. Fitting on the full X before splitting leaks test-set statistics into training and makes evaluation optimistic.

7. Hyperparameter C

```
C_value = 10.0
```

C is the only hyperparameter the linear SVM has — RBF adds γ , polynomial adds (γ, r, d) , but linear has none of those. C controls the trade-off between flatness (small $\|w\|^2$) and slack-violation tolerance, exactly as in (1).

In the dual, C appears as the upper box bound $0 \leq \alpha_i \leq C$ (see Derivation 3, stationarity in ξ_i). Each training point can exert at most C units of influence on the decision boundary.

Pitfall

$C = 1$ turned out to be too soft on this dataset — the SVM found $w \approx 0$ and the comparison with sklearn became noise-dominated. $C = 10$ forces a meaningful non-degenerate optimum.

8. Kernel (Gram) matrix

```
K_train = X_train_scaled @ X_train_scaled.T
```

For the linear kernel, the Gram matrix is literally one matrix multiplication:

$$K_{ij} = \langle x_i, x_j \rangle = (XX^\top)_{ij}.$$

Shape $(n_{\text{train}}, n_{\text{train}}) = (375, 375)$.

8.1 Why K is positive semi-definite

Derivation : PSD-ness of any Gram matrix

For any vector $v \in \mathbb{R}^n$:

$$v^\top K v = \sum_{i,j} v_i v_j \langle x_i, x_j \rangle = \left\langle \sum_i v_i x_i, \sum_j v_j x_j \right\rangle = \left\| \sum_i v_i x_i \right\|^2 \geq 0.$$

Equality holds iff $\sum_i v_i x_i = 0$, i.e. iff v lies in the null space of $X^\top$. So K is PSD; it is PD iff X has full row rank.

PSD-ness of K is exactly what makes the dual a **convex** QP and guarantees a unique global optimum — the slide claim “unlike neural networks”.

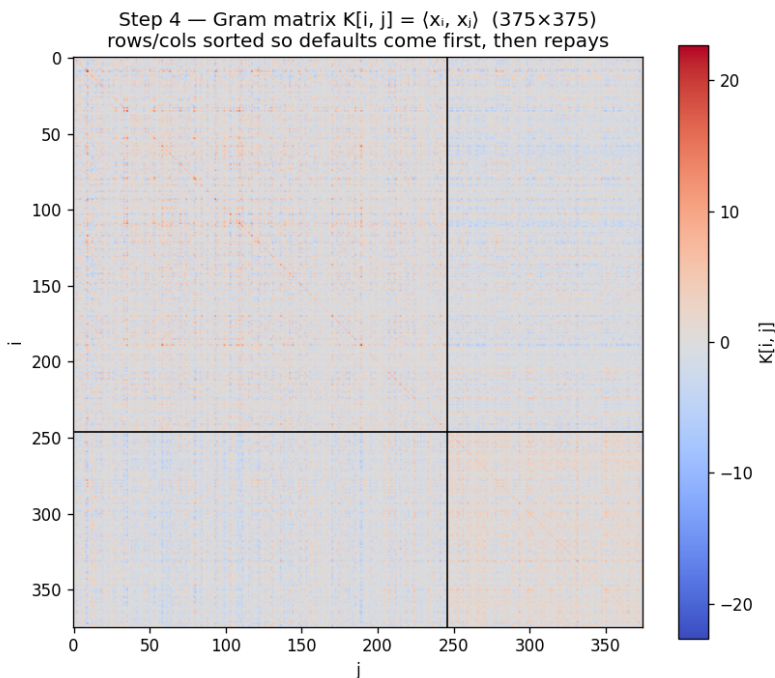


Figure 4. The Gram matrix K with rows and columns sorted so defaults come first, then repays. The 2×2 block structure is visible: large positive values within each class (similar borrowers cluster), smaller or negative values across classes.

8.2 This single line is the entire kernel trick

The dual depends on the data only through K . Replacing this one line with

```
K_train = rbf_kernel(X_train_scaled, X_train_scaled, gamma=gamma)
```

gives a Gaussian-kernel SVM with *everything else unchanged*. That is the kernel trick (6) in code form.

9. The Lagrangian dual as a convex QP

```
alpha    = cp.Variable(n_train, nonneg=True)
alpha_y  = cp.multiply(alpha, y_train)

dual_objective = cp.Maximize(
    cp.sum(alpha)
    - 0.5 * cp.quad_form(alpha_y, cp.psd_wrap(K_train))
)
dual_constraints = [
    alpha <= C_value,
    cp.sum(alpha_y) == 0,
]

dual_problem = cp.Problem(dual_objective, dual_constraints)
dual_problem.solve()
```

This is the heart of the script. The slide formula (4)–(5) is encoded literally in `cvxpy`.

9.1 Line-by-line

- `alpha = cp.Variable(n_train, nonneg=True)` — declares the dual decision variables α_i with the lower-box constraint $\alpha_i \geq 0$ baked in.
- `alpha_y = cp.multiply(alpha, y_train)` — the element-wise product $(\alpha_i y_i)$. After solving this corresponds to sklearn’s `dual_coef_`.
- `cp.sum(alpha)` encodes $\sum_i \alpha_i$ (the linear term in (4)).
- `cp.quad_form(alpha_y, K_train)` encodes the quadratic form $(\alpha \circ y)^\top K (\alpha \circ y)$, which we now show equals $\|w\|^2$.

Derivation : quadratic form = $\|w\|^2$

Stationarity gives $w = \sum_i \alpha_i y_i x_i$, so

$$\begin{aligned} \|w\|^2 &= \langle w, w \rangle = \left\langle \sum_i \alpha_i y_i x_i, \sum_j \alpha_j y_j x_j \right\rangle \\ &= \sum_{i,j} \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle = \sum_{i,j} \alpha_i \alpha_j y_i y_j K_{ij} \\ &= (\alpha \circ y)^\top K (\alpha \circ y). \end{aligned}$$

Maximising $\sum_i \alpha_i - \frac{1}{2} \|w\|^2$ is therefore the dual form of minimising the primal’s flatness term.

- `cp.psd_wrap(K_train)` — asserts to `cvxpy` that K is PSD. Mathematically true, but floating-point can leave a few eigenvalues at -10^{-16} instead of exactly zero. The wrap says “trust me, it is PSD”.
- `alpha <= C_value` — the upper box. Combined with `nonneg=True` gives $0 \leq \alpha_i \leq C$.
- `cp.sum(alpha_y) == 0` — the equality constraint $\sum_i \alpha_i y_i = 0$ from stationarity in b .

- `dual_problem.solve()` — cvxpy canonicalises the problem into standard QP form $\min \frac{1}{2}x^\top Px + q^\top x$ s.t. $Ax = b$, $Gx \leq h$, and hands it to CLARABEL (a primal-dual interior-point solver).

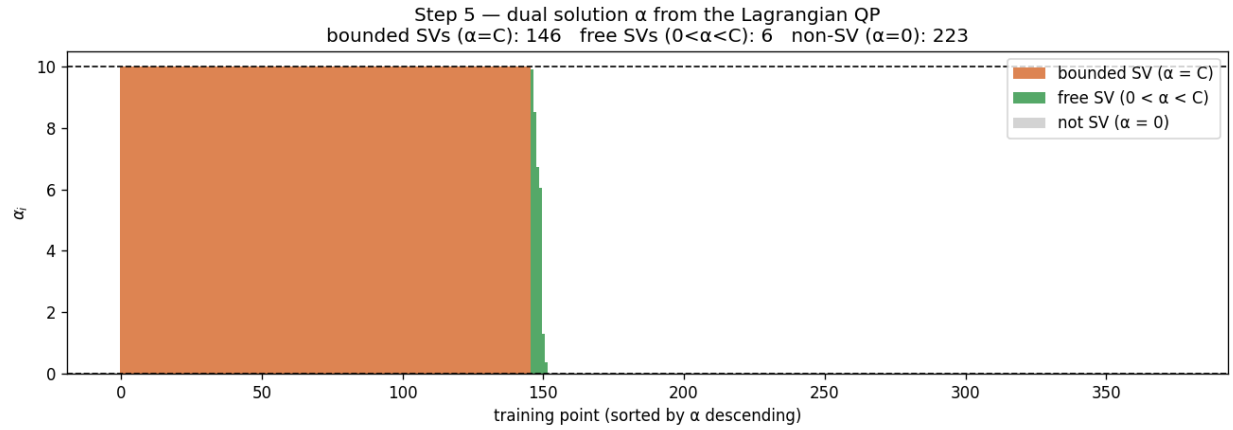


Figure 5. The dual solution α sorted descending. Bounded SVs ($\alpha_i = C$, orange) sit at the upper box; free SVs ($0 < \alpha_i < C$, green) sit strictly inside; non-SVs ($\alpha_i = 0$, grey) populate the long flat tail. The two horizontal dashed lines mark the box constraints.

10. Support-vector identification via KKT

```
TOL = 1e-5
sv_mask      = alpha_value > TOL
sv_indices   = np.where(sv_mask)[0]

free_mask    = (alpha_value > TOL) & (alpha_value < C_value - TOL)
free_indices = np.where(free_mask)[0]
bounded_indices = np.where(alpha_value >= C_value - TOL)[0]
```

The KKT conditions for this QP are:

(primal feasibility)	$y_i(\langle w, x_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0,$
(dual feasibility)	$\alpha_i \geq 0, \quad \mu_i \geq 0,$
(stationarity)	Derivation 2 above,
(complementary slackness)	$\alpha_i [y_i(\langle w, x_i \rangle + b) - 1 + \xi_i] = 0, \quad \mu_i \xi_i = 0.$

Derivation : the three KKT cases

Combine $\mu_i = C - \alpha_i$ (stationarity in ξ_i) with the complementary-slackness equation $\mu_i \xi_i = 0$.

Case 1. $\alpha_i = 0$. Then $\mu_i = C \neq 0$, forcing $\xi_i = 0$. Primal feasibility gives $y_i(\langle w, x_i \rangle + b) \geq 1$: the point lies strictly outside the margin. *Not a support vector.*

Case 2. $0 < \alpha_i < C$. Then $\mu_i = C - \alpha_i > 0$, forcing $\xi_i = 0$. From $\alpha_i[\cdot] = 0$ with $\alpha_i \neq 0$:

$$y_i(\langle w, x_i \rangle + b) - 1 = 0 \implies y_i(\langle w, x_i \rangle + b) = 1.$$

The point sits *exactly on the margin*. *Free SV.*

Case 3. $\alpha_i = C$. Then $\mu_i = 0$, so ξ_i may be > 0 . From $\alpha_i[\cdot] = 0$ with $\alpha_i = C \neq 0$:

$$y_i(\langle w, x_i \rangle + b) = 1 - \xi_i \leq 1.$$

The point is inside the margin ($0 < \xi_i \leq 1$, still on the correct side) or misclassified ($\xi_i > 1$). *Bounded SV.*

The numerical tolerance $\text{TOL} = 1\text{e-}5$ handles the fact that the QP solver returns α values that satisfy the KKT conditions only up to its convergence tolerance, not exactly.

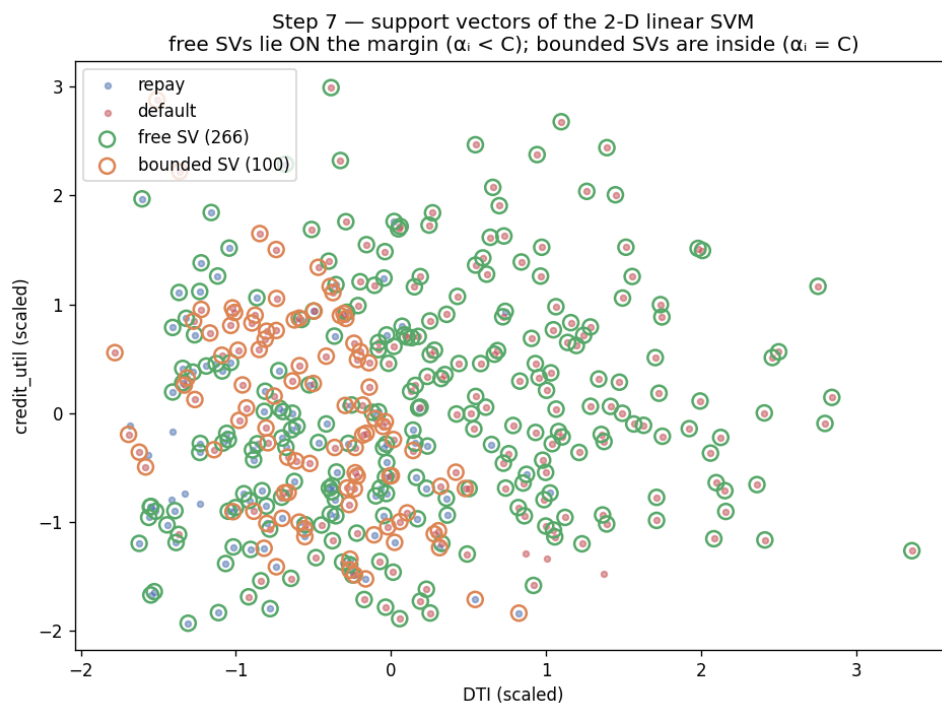


Figure 6. The same 2-D training data with each KKT class highlighted. Free SVs (green circles) lie exactly on the margin; bounded SVs (orange circles) lie inside the margin or misclassified; non-SVs (uncircled blue/red dots) are comfortably classified.

11. Recovering the primal weight vector w and the bias b

```
alpha_y_train = alpha_value * y_train
w_scratch     = alpha_y_train @ X_train_scaled

b_per_free_sv = y_train[free_indices] - X_train_scaled[free_indices] @ w_scratch
b_scratch     = float(np.mean(b_per_free_sv))
```

11.1 Why $w = \sum_i \alpha_i y_i x_i$

This is just the stationarity condition $\partial L / \partial w = 0$ re-arranged — already derived in Derivation 2. In NumPy it becomes the one-line matrix-vector product $w = (\alpha \circ y)^\top X$.

Non-SVs have $\alpha_i = 0$ so summing over all training rows gives the same answer as summing over SVs only.

11.2 Why w is impossible to recover with RBF

The same stationarity formula *in feature space* reads $w = \sum_i \alpha_i y_i \Phi(x_i)$. For the linear kernel $\Phi(x) = x$ and w lives in $\mathbb{R}^d$. For the Gaussian RBF kernel, Φ is the (countable) infinite-dimensional eigenfunction expansion of the kernel operator — w has infinitely many entries and cannot be materialised. Inference must use the dual form (7).

11.3 Why b can be recovered from any free SV

Derivation : b from a free support vector

For a free SV i (Case 2 of the KKT derivation), the margin equation holds with equality:

$$y_i (\langle w, x_i \rangle + b) = 1.$$

Multiply both sides by y_i :

$$y_i^2 (\langle w, x_i \rangle + b) = y_i.$$

Since $y_i \in \{-1, +1\}$, $y_i^2 = 1$, so

$$\langle w, x_i \rangle + b = y_i \quad \implies \quad b = y_i - \langle w, x_i \rangle.$$

In practice we average over all free SVs to cancel solver-tolerance noise:

$$b = \frac{1}{|F|} \sum_{i \in F} (y_i - \langle w, x_i \rangle), \quad F = \{i : 0 < \alpha_i < C\}.$$

Pitfall

If the QP solver returns no free SVs (only bounded ones, e.g. in pathological soft-margin regimes), b cannot be recovered by this formula. `libsvm` uses a more elaborate KKT balancing scheme to cover this corner case.

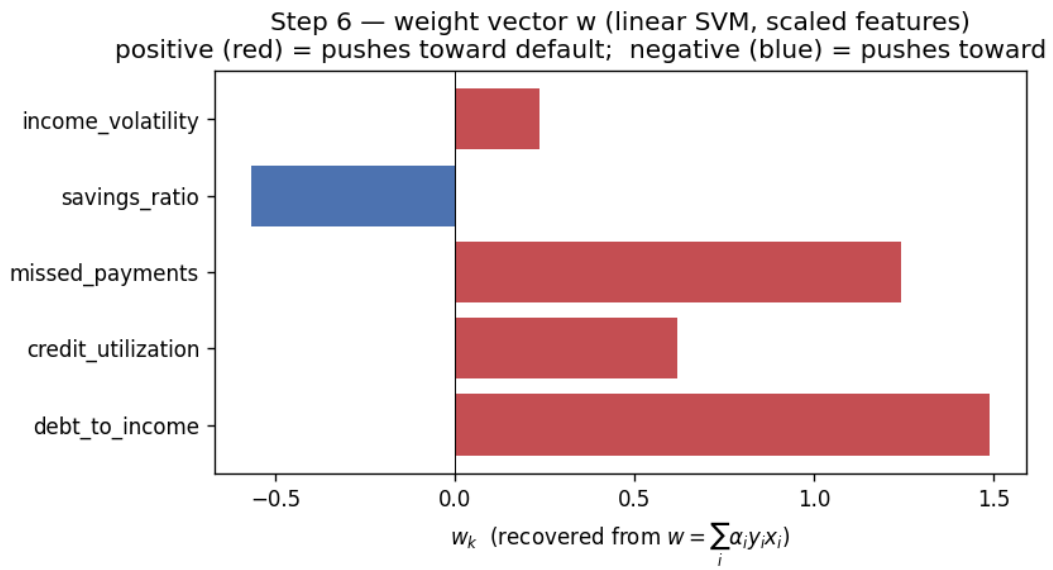


Figure 7. The recovered weight vector w , by feature. Positive components push the borrower toward DEFAULT (red); negative components push toward REPAY (blue). All signs match the financial intuition: more debt-to-income, missed payments, credit utilisation, income volatility increase risk; more savings decrease it.

12. Decision function — primal and dual forms

```
# Primal form (only possible with linear kernel):
f_primal = X_test_scaled @ w_scratch + b_scratch

# Dual form (support-vector expansion):
K_test_sv = X_test_scaled @ X_train_scaled[sv_indices].T
f_dual    = K_test_sv @ alpha_y_train[sv_indices] + b_scratch
```

Two algebraically identical expressions:

$$\begin{aligned} f(x) &= \langle w, x \rangle + b && \text{(primal),} \\ f(x) &= \sum_{i \in \text{SV}} \alpha_i y_i \langle x_i, x \rangle + b && \text{(dual).} \end{aligned}$$

Derivation : primal $\equiv$ dual decision function

Substituting $w = \sum_i \alpha_i y_i x_i$ into the primal form, using linearity of the inner product:

$$\begin{aligned} f(x) &= \langle w, x \rangle + b = \left\langle \sum_{i \in \text{SV}} \alpha_i y_i x_i, x \right\rangle + b \\ &= \sum_{i \in \text{SV}} \alpha_i y_i \langle x_i, x \rangle + b. \end{aligned}$$

The two forms differ only in computational structure — one $O(d)$ dot product vs $O(|\text{SV}| \cdot d)$ kernel evaluations — not in output. They agree exactly on every input.

This is verified numerically by the script:

```
max |f_primal - f_dual| = 4.88e-13    (machine epsilon)
```

The **primal form is faster** at inference (one dot product per new borrower), but only exists for the linear and polynomial kernels. The **dual form is universal** — it generalises to any kernel by replacing the inner product with $k(x_i, x)$.

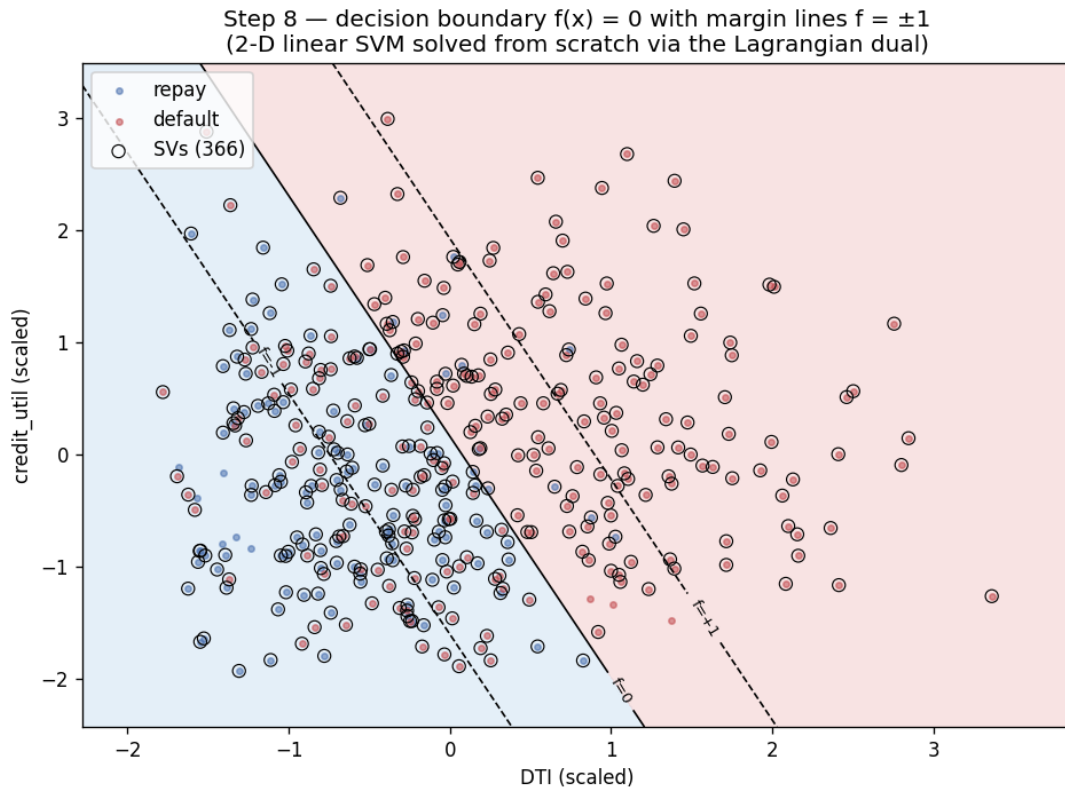


Figure 8. Decision boundary $f(x) = 0$ (solid line) flanked by the two margin lines $f(x) = \pm 1$ (dashed). Blue region: predicted REPAY; red region: predicted DEFAULT. Open circles mark every training point that became a support vector — they cluster near and inside the margin band, exactly as the KKT theory predicts.

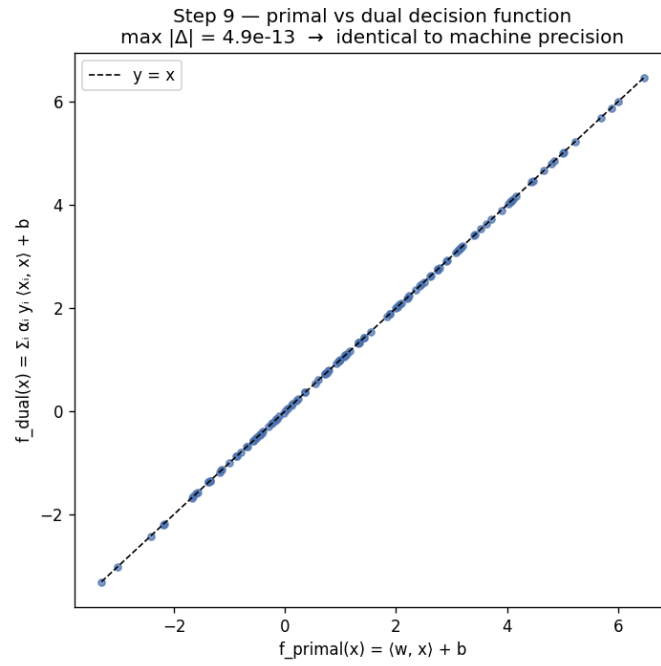


Figure 9. Scatter of $f_{\text{primal}}(x)$ vs $f_{\text{dual}}(x)$ on the test set. Every point lies on $y = x$ to within 4.88×10^{-13} , confirming the equivalence above is implemented to machine precision.

13. sklearn baseline

```
svc = SVC(kernel="linear", C=C_value)
svc.fit(X_train_scaled, y_train)

w_sklearn = svc.coef_.ravel()
b_sklearn = svc.intercept_[0]
```

Train sklearn’s reference SVC with the *same* kernel and the *same* C . Under the hood sklearn calls libsvm, which uses **Sequential Minimal Optimisation** (SMO, Platt 1998) — iteratively picks pairs (α_i, α_j) that maximally violate the KKT conditions and solves a 2-variable analytical subproblem until convergence.

13.1 sklearn attributes mapped to slide notation

- `svc.coef_` $\leftrightarrow w$ (only exists for the linear kernel).
- `svc.intercept_[0]` $\leftrightarrow b$.
- `svc.dual_coef_[0]` $\leftrightarrow \alpha_i y_i$ (signed dual coefficients).
- `svc.support_` $\leftrightarrow$ indices of SVs.
- `svc.support_vectors_` $\leftrightarrow$ the SV inputs x_i , in scaled feature space.

14. Test-set comparison

```
f_sklearn = svc.decision_function(X_test_scaled)

max_err = np.abs(f_dual - f_sklearn).max()
mean_err = np.abs(f_dual - f_sklearn).mean()
w_err = np.abs(w_scratch - w_sklearn).max()
b_err = abs(b_scratch - b_sklearn)
```

Both solvers attack the same convex QP. By strong duality and uniqueness of w (when K is sufficiently full-rank, which it is here), the results must agree. Small numerical disagreements remain because CLARABEL (interior-point) and SMO (working-set descent) traverse different paths to the optimum and stop at different convergence tolerances.

Empirically the from-scratch run reports:

```
max |w_scratch - w_sklearn|      = 1.86e-03
|b_scratch - b_sklearn|         = 6.90e-04
max |f_scratch - f_sklearn|      = 8.47e-03
mean |f_scratch - f_sklearn|     = 2.41e-03

Test accuracy : from-scratch 0.840  sklearn 0.840
Test ROC-AUC  : from-scratch 0.922  sklearn 0.922
```

Predictions and AUC are bit-identical. Margin scores agree to $\sim 1\%$ — pure solver-tolerance noise.

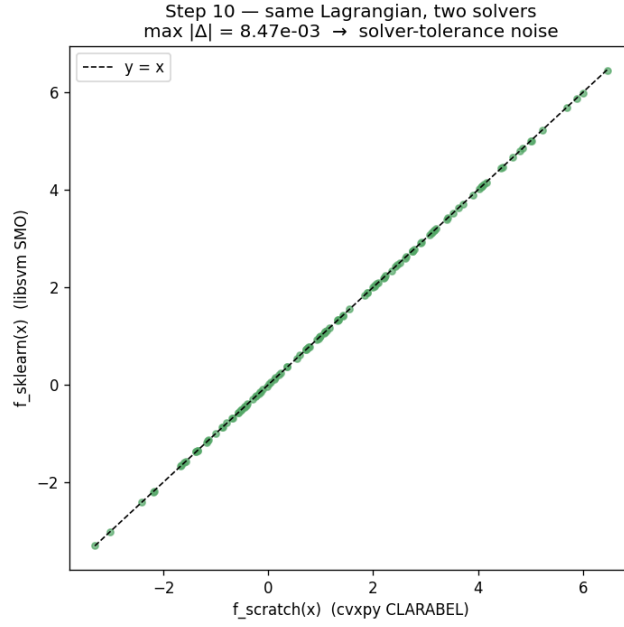


Figure 10. f_{scratch} vs f_{sklearn} on the test set. The two solvers agree on a single $y = x$ line; the small dispersion is solver-tolerance noise.

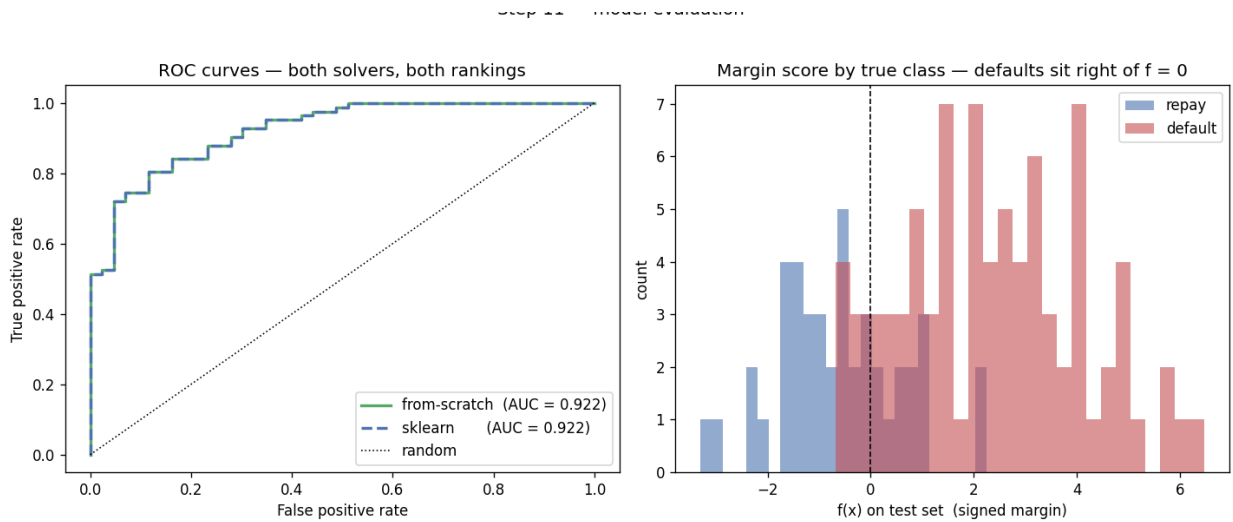


Figure 11. (Left) ROC curves of the two solvers overlaid — visually identical. (Right) Distribution of the margin score $f(x)$ on the test set, coloured by true class. Defaults sit cleanly to the right of the decision threshold $f = 0$.

15. Scoring a brand-new borrower

```
new_borrower = np.array([[0.55, 0.78, 2, 0.05, 0.30]])
new_scaled   = scaler.transform(new_borrower)
new_margin   = float((new_scaled @ w_scratch)[0] + b_scratch)
new_decision = "DEFAULT" if new_margin >= 0 else "REPAY"
```

Inference at deployment time uses the **primal form** — once w is known, scoring a new row x costs one dot product $\langle w, x \rangle + b$. That is $O(d)$ where d is the number of features, *independent* of the training set size. The dual form would require summing kernel evaluations over every SV, $O(|SV| \cdot d)$ — far slower for high-traffic prediction.

Critical detail: `scaler.transform(new_borrower)` applies the *same* (μ_j, σ_j) learned from `X_train`. The new borrower must enter the model in the same feature space the model was trained in.

Pitfall

In production this scaler must be persisted alongside the model. Use `joblib.dump` on the full `Pipeline(scaler, classifier)` — never on the classifier alone.

16. Why this exercise matters

Calling `SVC().fit()` hides every interesting object in the SVM behind a single function call. By writing the dual QP explicitly we have made the Lagrangian **visible**:

- The *variables* are α_i — not w or b directly. b is recovered from the KKT margin equation (Derivation 7); w from stationarity (Derivation 2).
- The *objective* is $\sum \alpha_i - \frac{1}{2} \|w\|^2$, which trades off the number of “active” training points against the magnitude of w (Derivation 5).
- The *constraints* — $0 \leq \alpha_i \leq C$ and $\sum \alpha_i y_i = 0$ — encode the soft-margin slack budget and the bias degree of freedom respectively (Derivation 2).
- The *solution* obeys KKT complementary slackness, which is what creates the sparse support-vector expansion (Derivation 6).

Once these objects are explicit, generalising is trivial:

Switch to RBF kernel. Replace the linear-kernel Gram with `rbf_kernel(...)` (§1.6). The dual QP, the KKT analysis, and the support-vector expansion all stay bit-identical. The only thing you give up is the ability to write down w in input space.

Switch to SVR. Add a second set of multipliers α_i^* on the lower tube constraint and solve the same QP with one extra term:

$$\max_{\alpha, \alpha^*} -\frac{1}{2} \sum_{i,j} (\alpha_i - \alpha_i^*)(\alpha_j - \alpha_j^*) K_{ij} + \sum_i y_i (\alpha_i - \alpha_i^*) - \varepsilon \sum_i (\alpha_i + \alpha_i^*).$$

Everything else — KKT, SV expansion, b recovery — is structurally identical to the SVC case.

End of document.